

Why Does Data Prefetching Not Work for Modern Workloads?

MAHMOOD NADERAN-TAHAN¹ AND HAMID SARBAZI-AZAD^{1,2*}

¹*Department of Computer Engineering, Sharif University of Technology, Azadi Av., Tehran, Iran*

²*School of Computer Science, Institute for Research in Fundamental Sciences (IPM), Tehran, Iran*

*Corresponding author: azad@sharif.edu

Emerging cloud workloads in today's modern data centers have large memory footprints that make the processor's caches to be ineffective. Since L1 data cache is in the critical path, high data cache miss rates degrade the performance. To fix the issue in traditional workloads, data prefetchers predict the needed data to hide the memory latency and ultimately improve performance. In this paper, we focus on the L1 data cache to answer the question on why state-of-the-art prefetching methods are inefficient for modern workloads in terms of performance and energy consumption? This is because L1 cache is the most important player affecting the processor performance. Results show that, on the one hand, these workloads suffer from low temporal locality and address repetition, and their consecutive accesses exhibit better spatial locality compared with the traditional workloads. On the other hand, miss patterns are spatially irregular and there is little opportunity to eliminate repetitive miss patterns. Because of these reasons, prefetching methods have poor performance and, with respect to more accesses to the lower memory levels, it is not energy efficient to use prefetchers for modern workloads.

Keywords: multicore architecture; hardware prefetching; scale-out workloads; workload characterization

Received 13 March 2015; revised 10 September 2015

Handling editor: Alan Marshall

1. INTRODUCTION

With the ever increasing volume of human and machine-generated data, big data and modern workloads¹ exhibit large amount of datasets that are split into shards and spread across multiple storage units. To close the gap between the core and the memory, multiple levels of caches are employed. The first-level data cache is primarily responsible for capturing the working set by keeping the most referenced memory addresses. However, the memory footprints of such workloads are large and data cache is not able to capture the whole referenced addresses. As a result, a fraction of accesses are missed in the cache.

The most straightforward approach to hide the memory latency and capturing such a large memory footprint is to use large caches. Although a large cache increases the hit rate, it suffers from high access latency. Figure 1 shows a survey of commercial micro-architectures that incorporate small data caches to maintain low access latency. Therefore, using a large

cache is not an appropriate technique. Another approach for hiding the memory latency is reducing cache misses via architectural designs, called *data prefetching*, while using small caches. With the help of repetitions and regulations observed in the memory access stream, it is possible to find patterns in the references by tracking the history and predict future addresses. This approach has been widely studied for compute-intensive and single-thread workloads, typically SPEC CPU [1–10]. Also in multi-core architectures, the prefetchers are evaluated with a set of multi-program workloads [11–18].

Traditional server workloads, which are basically multi-threaded, feature spatial correlation in the memory access stream. By tracking the memory references in large regions consisting of multiple cache blocks, it is possible to cover 30–60% of L1 misses [19, 20]. However, the pros and cons of prefetching methods for modern workloads have received little attention. For example, without mentioning the reason, [21] showed that disabling Next-Line prefetcher has no impact on the performance.

This paper is a study on modern workloads to evaluate different aspects of prefetching methods. The focus of this work

¹ In this paper, we use big data, modern, cloud and scale-out workloads interchangeably since their differences do not affect this work.

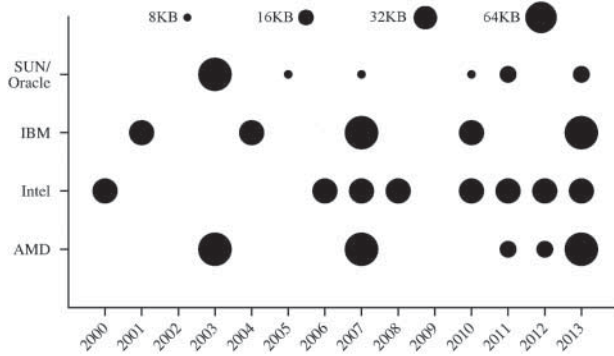


FIGURE 1. L1 data cache size trend over the past decade. While larger L1 data cache is desired in emerging applications, processor manufactures do not tend to push up its size because of latency constraints.

is the L1 data cache since L1 cache has a crucial effect on system performance. While the L1 instruction cache has high hit ratio for most of the modern workloads and L2 is able to capture the instruction working sets, L1 data cache exhibit high miss rate [21]. Since L1 data cache is the first place for accessing data blocks, such high miss rates are important sources of poor performance. We first show that current state-of-the-art prefetching methods have low performance. Next, we conduct a series of locality analysis on programs memory references as well as cache miss analyses and show that there exist key differences between traditional and modern workloads. In traditional workloads the opportunity for efficient prefetching is high because misses are spatially predictable or temporally repeatable. However, most of the cloud workloads have irregular and unpredictable miss patterns in memory space and there is little opportunity for temporally eliminating repetitive miss patterns. With such a knowledge, we show that prefetching methods are not energy efficient since they increase costly accesses to lower levels of memory much higher than the achieved performance.

Briefly, the empirical evaluations of this work show:

- (i) Existing history-based prefetchers that were originally proposed for desktop and server workloads are quite inefficient in reducing cache misses. The main problem is high overprediction rates that means prefetched blocks are evicted without any access and cache is filled with useless blocks.
- (ii) Modern workloads feature poor temporal locality and there are large number of accesses with few repetitions. Therefore, there are little opportunities for address correlation prefetchers. Reuse distance analysis shows that unlike other workloads, there are variable reuses at different distances. This feature is similar to traditional workloads with high miss rates.
- (iii) Near neighbor accesses of scale-out workloads have strong spatial locality even more compared to other applications.

TABLE 1. Main characteristics of the simulated system. The latencies assume 45 nm process at 2.5 GHz.

Processor	UltraSPARC-III ISA, 8-stage pipeline, out of order, 4-wide dispatch/retirement, 96-entry ROB, 32-entry LSQ, 16-entry store buffer, 2.5 GHz, 45 nm
L1 caches	Split I/D, 64KB private, 2-way, 64B, LRU, write-through, 2-cycle hit latency, 32-entry MSHR
L2 cache	8 MB shared, unified, non-inclusive, 8-way, 64B, LRU, write-back, 10-cycle hit latency, 64-entry MSHR (instruction and data)
Main memory	64 GB, 60 ns access latency, 4 memory controllers

- (iv) There is no regularity for cache misses even in large memory regions. Even that some workloads exhibit a degree of regularity, miss patterns are complex.
- (v) There is little opportunity for eliminating temporal repetitive misses because a large fraction of misses are non-repetitive.
- (vi) In cache-friendly applications, top reused consecutive deltas mostly result in hit and in prefetch-friendly applications, they symmetrically hit or miss in cache. However, in scale-out workloads they randomly hit/miss in the cache; hence delta correlation prefetchers wrongly predict future addresses.
- (vii) Prefetchers are not energy efficient because with the increase in the number of memory requests, more costly accesses are made to the lower memory levels.

The rest of this paper is organized as follows: Section 2 explains the methodology and workloads. Section 3 evaluates the efficiency of well-known prefetchers. In Section 4, locality analyses are explained. Section 5 presents cache analysis. Section 6 discusses the energy aspects of using prefetchers for scale-out workloads. Section 7 presents the related work and Section 8 summarizes this study.

2. EVALUATION METHODOLOGY

The focus of this paper is the data cache and for evaluations we use trace-driven and full system simulations of a multi-core processor using FLEXUS [22]. We simulate a 16-core directory based shared L2 multi-core system, which is common in today's server market, running *Solaris 9*. The list of simulation parameters are presented in Table 1.

Through this paper, we compare these workloads with a set of reference workloads. The reference workloads are selected based on some metrics that are described below.

TABLE 2. Characteristics of workloads. In addition to Cloudsuite, some traditional desktop and server workloads have been selected. In the analyses, *Vips* and *milc* are considered as cache-friendly and others have the same or higher miss rates than Cloudsuite. MPKI represents number of misses in L1 data cache per 1K instructions.

	Workload	Description	Input	Miss ratio (%)	MPKI
Cloudsuite	Data serving	Key-value NoSQL data store	Cassandra, 15 GB YCSB dataset	5.66	7.06
	Map-Reduce	Large scale analysis	Bayesian classification algorithm, 4.5 GB set of Wikipedia pages	6.39	7.44
	Web frontend	Web services for availability	Cloudstone, 12 GB on-disk file dataset	2.59	4.70
	SAT solver	Software testing	Cloud9 symbolic execution, command-line <i>printf</i>	3.2	4.40
	Web search	Index Serving Node	Nutch/Lucene, 230 clients, 1.4 GB index, 15 GB data segment	2.47	4.38
	Media streaming	Video streaming server	Darwin Streaming Server, 7500 clients, 60 GB data set	5.73	7.92
Traditional	Vips	Image processing	1 image, 2662 × 5500 pixels	0.15	0.3
	Libquantum	Quantum computing	reference input	12.2	11.3
	lbm	Computational Fluid Dynamics	reference input	22	52.7
	milc	Quantum Chromodynamics	reference input	0.5	1.8
	Soplex	Simplex Linear Program (LP) Solver	reference input (pds)	8.9	24.6
Server	hmmmer	Search a gene sequence database	reference input (nph)	2.5	5.6
	Oracle	Online Transaction Processing	100 warehouses (10 GB), 16 clients, 1.4 GB SGA	6.4	11.7
	Apache	Web Server	16K connections, FastCGI, worker threading model	5.8	11.6

- (i) *Cache friendliness*: This type of workloads have low miss rates at the first level data cache. As a result, temporal and/or spatial localities exist in the stream of memory accesses and cache captures the working set of the applications. We call a workload to be cache-friendly if the miss rate is less than 0.5% or the number of misses per kilo instruction is less than 1.
- (ii) *Prefetch friendliness*: This type of workloads have typically high miss rates; hence they lack either temporal or spatial (or both) localities. However, a large fraction of misses are covered with existing prefetchers by finding the correlations among previous accesses. We call a workload to be prefetch-friendly if there exists at least one history-based method that can cover at least 50% of misses.

Table 2 shows the list of benchmarks with their parameters. Cloudsuite is a benchmark suite of emerging and modern scale-out workloads which consists of six benchmarks [21]. For comparison, we use some traditional desktop and server workloads with different properties. We select *Vips* from PARSEC suite [23] and *milc* from SPEC CPU2006 [24] because they have high hit rates (99.85 and 99.5%, respectively), therefore they are cache-friendly. Other workloads including *libquantum*, *lbm*, *soplex*, *hmmmer*, *Oracle* and *Apache* have the same or higher miss rates than Cloudsuite. For example, the miss rate of *lbm* is 22% which is the highest in our selection.

3. EVALUATION OF PRIOR PREFETCHERS

In order to see the ability of current history-based data prefetchers for reducing cache misses, we have selected some state-of-the-art methods that were originally proposed for both SPEC CPU and server workloads. We have considered a number of features for selecting the methods including the popularity, publication data and performance (specifically the coverage). Furthermore, the selected methods are able to track the history in both time and space dimensions. We have not selected throttling methods² because, as we discuss later, the low coverages achieved by prefetching methods are not related to pollution.

NLP: Next-Line-Prefetching method is the simplest and earliest method that on every access to a block, X , issues a prefetch request for the next block, $X + 1$.

Stream: The concept of stream prefetching has been presented in [8, 25–27]. This method is capable of tracking multiple access streams and for each of them, a tracking entry is created in the table which can be one of four different states: (1) *Invalid*: Initially all tracking entries are in this state. (2) *Allocated*: Upon a demand miss, the block address is allocated in a table if there is no matching entry for that. (3) *Training*: The prefetcher trains the direction of the stream based on the next two misses that occur ± 16 cache blocks from the first miss

² Such methods increase/decrease the prefetch degree based on some metrics including cache pollution in the presence of prefetcher.

and (4) *Monitor and Request*: The tracking entry monitors the accesses to a memory region from a *start pointer* (A) to an *end pointer* (P). The maximum distance between the start and end pointers is determined by *Prefetch Distance*. If there is a demand access to a block in the monitored region, the prefetcher issues cache blocks $[P + 1, \dots, P + N]$ for ascending direction ($[P - 1, \dots, P - N]$ for descending directions), where N is called the *Prefetch Degree*. Next, the tracking entry moves the monitoring region and starts to monitor the region between $A + N$ to $P + N$. In our implementation, we configure the aggressiveness in the middle-of-the-road with a prefetch distance of 16 and a prefetch degree of 2.³

GHB: Global History Buffer is an n -entry FIFO table that keeps track of the n most recent accesses [4]. Each entry stores a pair of an address and a link pointer and entries are chained in a time-ordered fashion. An index table holds the initial pointers to the GHB entries which can be indexed in various ways. As a result, different history-based methods can be implemented. Usually the index table tracks the PCs (program counter) to localize the accesses. The GHB table is responsible for tracking the history and extracts correlations among previous accesses. In the delta correlation (DC) implementation, every two chained entries are assigned with a delta value. With a predefined prefetching degree, d , the last d entries in the GHB form the pattern and GHB is searched to find the last occurrence of the pattern. Once it is found, the next d deltas are used to generate new addresses. The index table and GHB in our simulations have 64 and 1K entries.

ISB: Irregular stream buffer is an extended implementation of the original GHB targeting irregular access patterns [18]. In this method, an extra level of indirection is used to create a new structural address space in which correlated physical addresses are assigned consecutive structural addresses. The prefetcher is composed of three components: (1) Training Unit that takes the load PC and load address and it maintains the last observed address in each stream. (2) Physical-to-Structural Address Mapping Cache (PS-AMC) that stores the mapping from physical address space to the structural address space and it is indexed by physical addresses and (3) Structural-to-Physical Address Mapping Cache (SP-AMC) that stores the inverse mapping of PS-AMC and it is used for extracting the correlated addresses in the stream upon a trigger access. For prediction, each cache access becomes a trigger access, causing the PS-AMC to retrieve the structural address of the trigger address. By searching the structural address in SP-AMC, the prefetcher issues the next addresses with the same structural address. The training unit in our simulations holds 64 entries while PS-AMC and SP-AMC contains 1K entries.

SMS: Originally developed for commercial server workloads, SMS exploits spatial correlations among data beyond cache blocks [19]. In this method, multiple consecutive blocks form a spatial region and the first access in the region is called

the trigger access. Within a time interval, all accesses in the spatial region are recorded. It uses three tables for training and predicting the blocks. There is a filter table that records the spatial region tag, PC and spatial region offset of the trigger access. Next matching accesses move the spatial patterns from filter to accumulation table. Each entry stores the PC and spatial region offset of the trigger access, and a spatial pattern bit vector indicating which blocks have been accessed during the generation. At the end of the generation, the spatial pattern is moved to Pattern History Table (PHT) for further predictions. Upon a trigger access, SMS consults the PHT for a matching entry and predicts the blocks within the region that have been recorded before. We configured the filter and accumulation tables to hold 64 entries while the PHT size is infinite (16K entries).

The two most important metrics to evaluate prefetchers are *coverage* and *overprediction*. By definition, coverage refers to the fraction of misses that are eliminated by the prefetcher and overprediction refers to the fraction of prefetched blocks that are evicted without any access. For an *efficient* prefetcher, we expect a high coverage and moderate overprediction; otherwise, it is considered as *inefficient*. Figure 2 shows the coverage and overpredictions for different methods. It is clear that none of the methods achieve a good coverage for scale-out workloads despite that all methods have been previously evaluated for their class. For example, nearly 90% of misses in *libquantum* and *lbm*, which have higher miss rates than scale-outs, are covered with next-line prefetching. However, it has low coverage (10–35%) for scale-out workloads. Moreover, SMS which has a distinguishable coverage for server workloads (especially *Oracle*) has low coverage for most of the scale-out workloads. For example, in *Data Serving*, *Map Reduce* and *SAT Solver*, the coverage is less than 15%. It is interesting to see that for scale-outs, the simple next-line prefetching has higher coverage than complex correlation-based methods. Nonetheless, the difference is not comparable with traditional workloads. The situation is worse for other methods (ISB and GHB) in a way that most of them increase the miss rate by wrongly predicting future addresses (negative coverage).

We select two of the methods, SMS and GHB PC/DC for more inspection because on average they operate better than other methods. Figure 3 shows the coverage and overprediction of SMS with different spatial region sizes. The numbers in the second row of x -axis show the total storage cost of SMS with the given spatial region size and infinite PHT size. While SMS with 2K regions can cover 40–60% of misses with the overprediction rate less than 20% [19], most of the workloads suffer from low coverages and high overpredictions. In *Data Serving*, *Map-Reduce*, *SAT Solver* and *Media Streaming*, the coverages are less than 30%. Specifically the coverages of *Data Serving* and *SAT Solver* are less than 15%, while the overpredictions are 40 and 60%, respectively. *Web Search* is the only exception where it has more than 30% coverage and less than 30% overprediction. Increasing the spatial region size, in order to capture more blocks, has little impact on the coverages except

³ We use the same terminology as in [8].

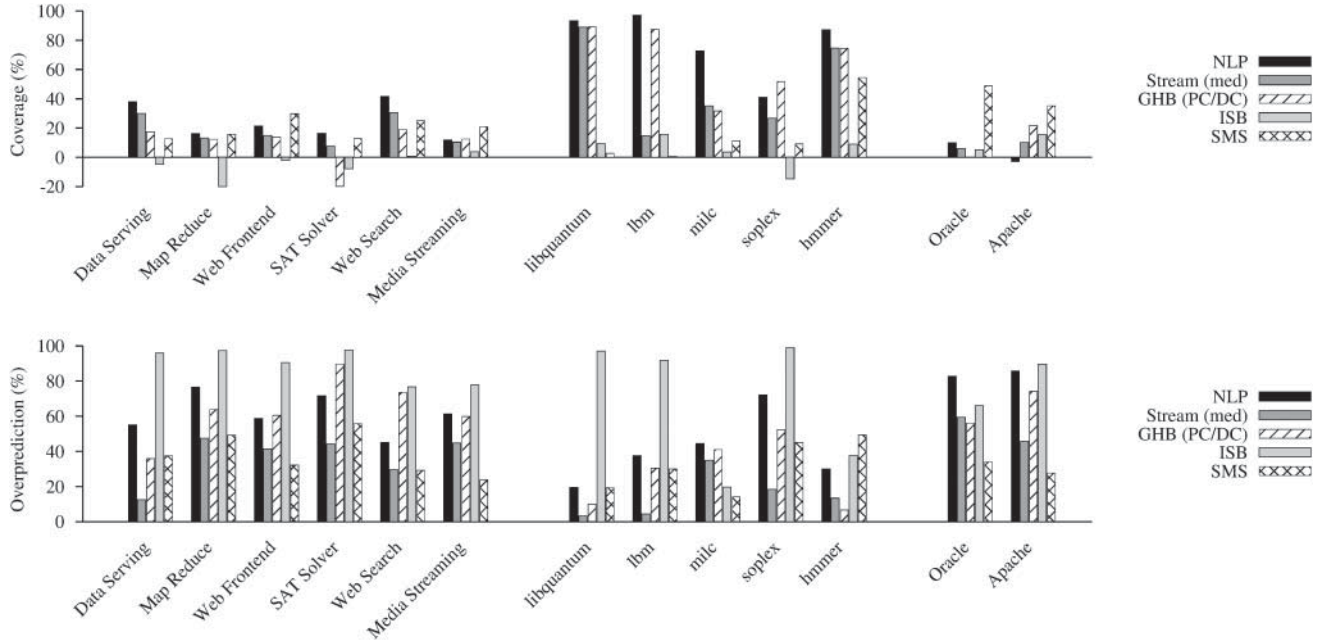


FIGURE 2. Prefetching coverage shows that neither SMS which has been proposed for server workloads nor other methods which have been proposed for SPECCPU workloads are efficient for scale-outs. In some cases, the coverage value is negative which means that the prefetcher increased the miss rate.

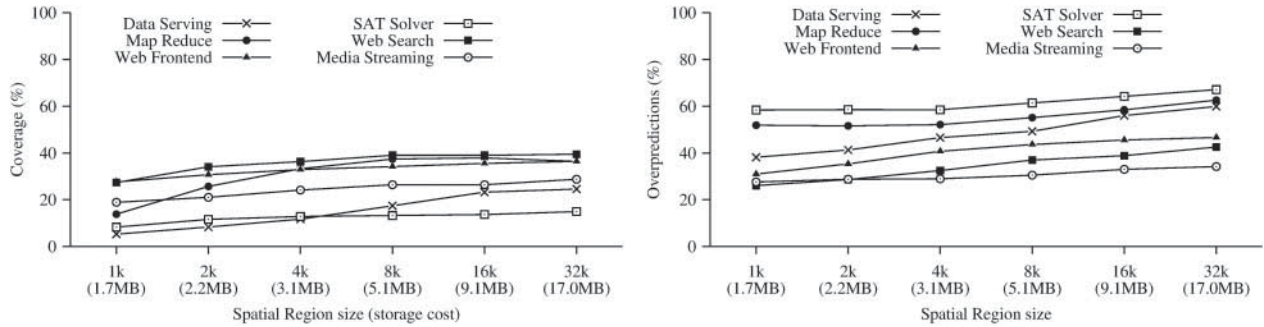


FIGURE 3. SMS coverage and overprediction. While SMS with 2K regions covers 30–40% of L1 misses of traditional server workloads, it has low coverage for Cloudsuite workloads. Furthermore, the overpredictions are high compared with server workloads. The numbers in the second row of X-axis labels show the total storage cost with the given spatial region size and infinite PHT (16K).

for *Map-Reduce* where a 20% jump has been observed but at the same time, the overpredictions increase. At 32K regions, where 512 blocks are recorded, all workloads suffer from high overpredictions.

Same metrics for GHB PC/DC are presented in Fig. 4. For the prefetch-friendly application such as *libquantum*, *lbm*, *soplex* and *hminer*, we have observed that GHB with 1K entries covers 90% of L1 cache misses with roughly 10% overprediction rate. However, for scale-out workloads, the results are far from expectation. Surprisingly, *SAT Solver* has negative coverages⁴ with all history sizes. Correspondingly, the overpredictions

are high for most of the workloads. By increasing the history size, the coverage of *Map-Reduce* decreases. That means the access patterns change over large history window; hence the predictions are incorrect and the high overprediction rate confirms it.

Our observations confirm that the number of issued blocks in scale-out workloads are less than issued blocks in traditional workloads. As a result, the low coverages and high overpredictions are not caused by pollution. Additionally, when a correlation-based method detects a pattern and issues some blocks for prefetching, the predicted information are not totally useless; instead they are used after multiple insertions and evictions. Applying filtering mechanisms to remove the overpredictions decrease the opportunity of speculations.

⁴ Negative coverage means that the prefetcher could not eliminate cache misses; instead, it increased the misses over the baseline architecture.

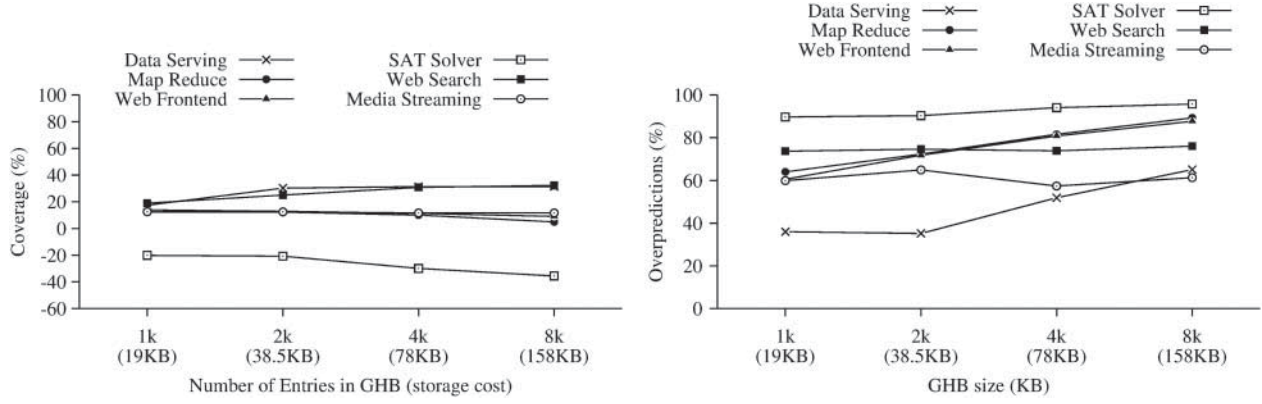


FIGURE 4. GHB coverage and overprediction. Although GHB covers 90% of L1 misses of a prefetch-friendly application, *Libquantum*, it has very low coverage for Cloudsuite workloads. Negative coverage means that the prefetcher increased the number of cache misses compared with the baseline architecture. Again, the numbers in the second row of X-axis labels show the total storage cost with the given history size.

4. LOCALITY ANALYSIS

In this section, we analyze the concepts of temporal and spatial localities. The purpose is to find the differences between scale-out and other types of workloads. Locality principals are easy to understand but hard to measure. Some terms such as ‘near future’, ‘nearby addresses’, ‘high probability’ are general concepts and there is no exact definition for them.

4.1. Temporal locality

Reuse distance is a popular approach for measuring cache locality and it has been widely studied since its introduction [28–32]. By definition, reuse distance is the number of distinctive elements accessed between two consecutive uses of the same element. Figure 5 shows an example and a breakdown of reuse distances. A reuse distance of zero means that there is no distinctive element between the occurrence of one element.

Figure 6 shows the breakdown of different ranges for scale-out and other workloads. The maximum distance in our analysis has been set to 8192 which resembles a 64KB cache. In *Vips*, a vast majority of distances are below 64 but other workloads exhibit different behaviors. For example, in *libquantum*, a great fraction of distances are beyond 8K and in *hmmmer*, the vast majority of distances are between 4K and 8K. Comparing Cloudsuite workloads with others show that there exists some similarities between both traditional and server workloads. In other words, a mix range of reuse distances exists not only in workloads with high miss rates but also in workloads with low miss rate such as *milc*.

The next analysis regarding the temporal aspect is to find out how repeated addresses are distributed in the access stream regardless of their distances. As an example, assume a sequence contains $\{A, B, C, A, C, E, D, B, A\}$. In this sequence, the first top reused address, *A*, contributes 33% of total accesses while

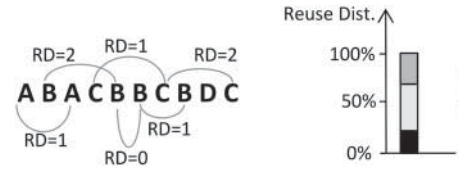


FIGURE 5. An example of reuse distance. Note that between the first and second instances of *C*, there is only one distinct symbol, *B*, while it is repeated twice.

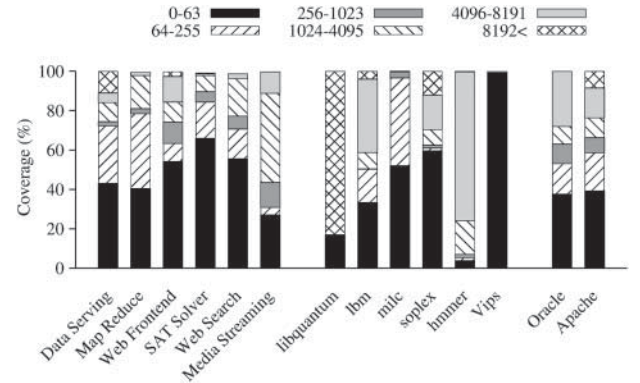


FIGURE 6. Scale-out workloads have variable and large reuse distances. There are also large distances with few repetitions.

second or third top reused addresses, *B* or *C*, contributes 22%. As a result, 77% of total repetitions comes from only 3 addresses which we call them top three reused addresses. For an application with good temporal locality, we expect a high repetition weight in few addresses. Implicitly that means, addresses are repeated in the sequence with high probability.

Figure 7 shows the result of this analysis where the X-axis show top 20 reused addresses in descending order. For example, the index of 1 means the contribution of the first top reused

address and the index of 2 means the cumulative contribution of first and second top reused addresses. Because of the logarithmic growth of contributions and to better show the results, we prefer to show the top 20 reused addresses. In *Vips*, nearly 80% of repetitions comes from only twenty addresses. Some traditional workloads have less contribution than Cloudsuite workloads. Specifically, *hmmmer* has very low contribution which means addresses repeat few times. In the scale-out

workloads, the top 20 reused addresses contribute less than 30% in all workloads. Specifically, *Web Search* and *Media Streaming* have very poor contributions. Although *libquantum* has the highest miss rate among selected workloads, top 20 reused addresses contribute more than most of the scale-out workloads.

From the two analyses it can be seen that these workloads exhibit repetitions in their access sequences; however, there are a large set of addresses with small repetitions and the repetitions repeat in large and variable distances. Such features can clarify why address correlation methods fail to predict correctly. The methods can predict a future address but most of the time the prediction is incorrect because the addresses have few repetitions. To make it more clear, assume $S1$ contains $\{A, A, A, B, B, B\}$ and $S2$ contains $\{A, A, B, B, C, C\}$. Both of them have repetitions but $S2$ contains more distinctive addresses which result in fewer repetitions. As a result, it is harder to predict future accesses for $S2$.

4.2. Spatial locality

Spatial locality refers to the high probability of access to nearby locations of X upon an access to X . The concept is easy to understand but there is no unique and standard method for the measurements. Therefore, every analysis leaves some corners uncovered. In our methodology, we only focus on consecutive accesses which is able to explain some key and basic features. To measure it for N accesses and block size of B bytes, a counter, C , increments whenever M consecutive addresses map to the same block. The degree of spatiality is then defined as $C/(N - 1)$. As an example, assume a memory sequence of $\{A1, A2, A3, A4, A5, A6\}$ where each address maps to $\{B0, B0, B1, B2, B2, B2\}$. Assuming that $M = 2$, the counter is incremented for $(B0, B0)$, $(B2, B2)$ and $(B2, B2)$. As a result, the degree of spatiality is 60%.

Figure 8 shows the spatial locality with different configurations. Each row shows the spatial locality for two and three consecutive accesses to model the locality. For each configuration, the block size is varied from 16B to 8KB. Cloudsuite

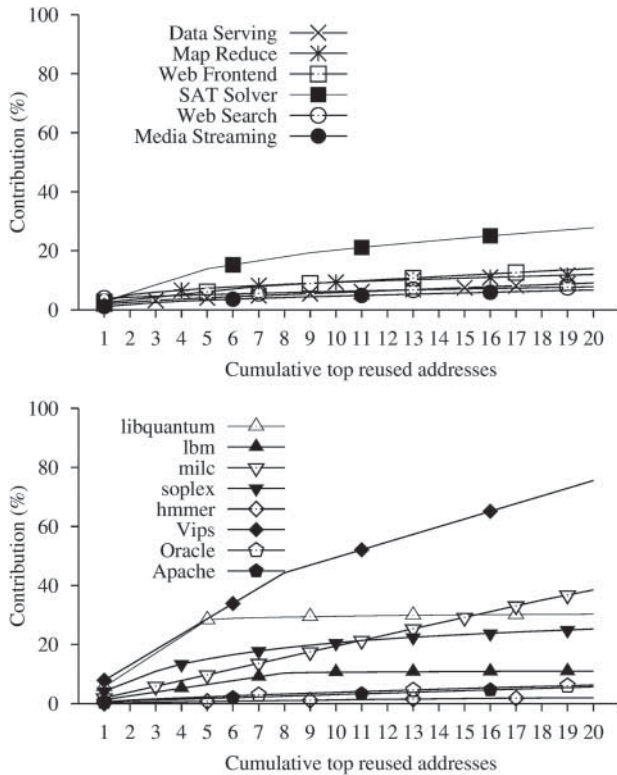


FIGURE 7. Distribution of repeated addresses. Addresses in most of the Cloudsuite workloads have low repetitions compare to *Vips*. It is interesting to see that although *libquantum* has higher miss ratio than Cloudsuite workloads, addresses are more repetitive.

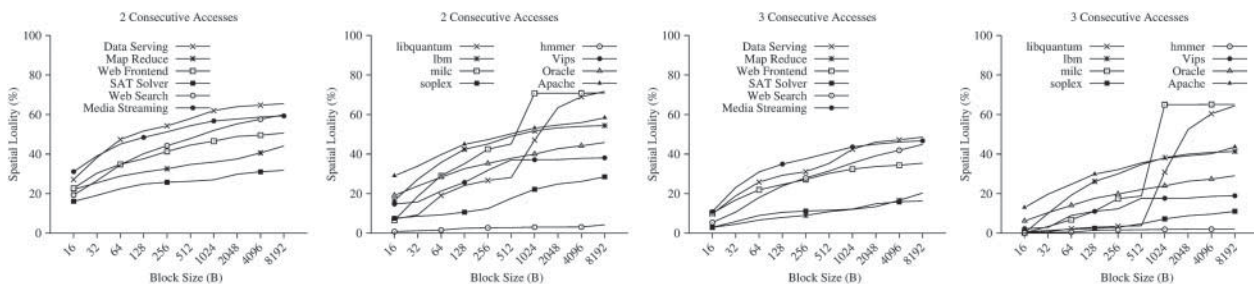


FIGURE 8. Measuring spatial locality of applications by changing the block size. Interestingly, most Cloudsuite workloads have higher spatial locality than conventional workloads. It is even higher for small block sizes.

workloads generally have higher degree of spatiality. Such feature is also seen in traditional server workloads but SPEC workloads have different behaviors. For example, *lbm* and *libquantum* have higher spatial locality than *soplex* and *hammer* but at the same time, they have higher miss rates. Two workloads, *soplex* and *hammer*, have the least degree of spatial locality although they have nearly the same miss rates compared with scale-out and server workloads.

From the locality analyses, we see that the some cache-friendly applications such as *Vips* benefits from temporal locality while *milc* benefits from spatial locality. Some of the prefetch-friendly applications suffers from both low temporal and spatial localities. They also have reuse distances, to some extent, similar to scale-out workloads. In Cloudsuite, address repetitions are low and when a correlation-based prefetcher predicts a new address, there is low probability for reusing that. Such behavior has been observed in SMS where it has low coverage for scale-outs although it primarily exploits spatial features.

Problem with reuse distance analysis is that the results can be verified with measuring the cache miss rates. In other words, reuse distance analysis shows that scale-out workloads have larger distances than *Vips* and smaller distances than *libquantum*. The reuse distances are also similar to server workloads. In fact, the measurements in Table 2 show that these workloads have misrates higher than *Vips* and lower than *libquantum* and similar to server workloads. The question that locality analyses are unable to answer is how cache misses are repeated in the stream. While address repetition is a software issue, miss repetition is a hardware issue which depends on the structure of cache. As a result, more in-depth analyses are required to determine how misses are repeated.

5. CACHE MISS ANALYSIS

In this part, we focus on cache misses where the repetitiveness of misses are explained. The observations are summarized at the end of this chapter to explain why data prefetching is hard in scale-out workloads.

5.1. Space dimension

In this section, we try to answer the question on how misses are repeatable? In other words, do they happen regularly to form patterns? To find it out, memory space is divided into relatively large chunks by increasing the block size from 64B to 8KB which is equal to one page. For each incoming miss, the tag is calculated by masking the lower 13 bits. If the tag number is new, a logical region number is assigned to it. Otherwise, the previously recorded region number is used. Finally, by plotting the region numbers based on the order of the misses in time, it is possible to view the recurring miss patterns. Figure 9a shows an example of this analysis where the memory space is

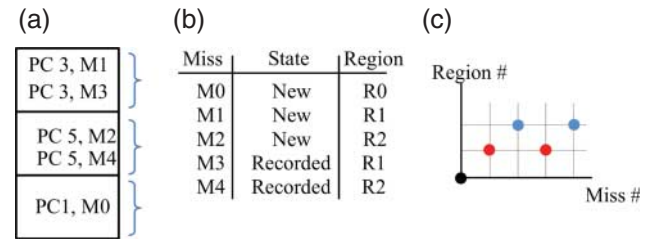


FIGURE 9. Example: analyzing the repetition of misses. (a) Memory space is divided into equal chunks. (b) New misses are assigned a logical number and (c) region numbers are plotted in the order of misses. Misses that belong to the same PC, have similar color. Horizontal lines show how misses repeat in regions. As a result, patterns can be detected.

divided into 8KB regions and the order of incoming misses are marked as M_i . Figure 9b shows how a region number is assigned to each miss. R_0 , R_1 and R_2 are assigned to M_0 , M_1 and M_2 , respectively, since have new tags. For the next miss, M_3 , the tag number has been recorded before, so R_1 is assigned to M_3 . The process continues for all misses and in Fig. 9c, the data points are plotted with X-axis denoting the miss numbers and the Y-axis denoting the region numbers. Since, each miss has a PC, we assign a color to misses with the same PC. This is useful to identify whether PC stream create regular miss patterns or not. In general, with horizontal lines it is possible to identify reused regions. Since the region numbers are assigned to new incoming misses only, the final plot has a triangular shape.

The result of this analysis for scale-out and traditional workloads are shown in Figs 10 and 11, respectively. Since the number of PCs are large in the traces, we only colorized the top 10 reused PCs. Specifically, misses for top reused PC is shown in red.

Figure 10 shows the result of this analysis for scale-out workloads and for each chart, a region is magnified. *Data Serving* has recurring patterns but there are a lot of noises that make the patterns hard to detect. *Map Reduce*, on the other hand, has patterns but magnifying the patterns shows that they are complex. Because of such patterns, a large history should be used in order to track the history. As shown in Fig. 3, at large memory regions, SMS is able to cover a fraction of misses. Unlike the first two workloads, other scale-outs contain random misses. Figure 11 shows the the result of this analysis on traditional workloads. Note that we ignored *Vips* because it has very low number of misses. Most of the SPEC workloads, such as *libquantum*, *lbm*, *milc* and *hammer*, have simple recurring patterns. Specifically, most of the misses in *lbm* and *libquantum* and, to some extent, *soplex* are caused by few PCs. *Oracle* has some short recurring patterns which is why SMS has the best coverage among other prefetchers. Nonetheless, *Apache* has a behavior similar to Web-related workloads in Cloudsuite.

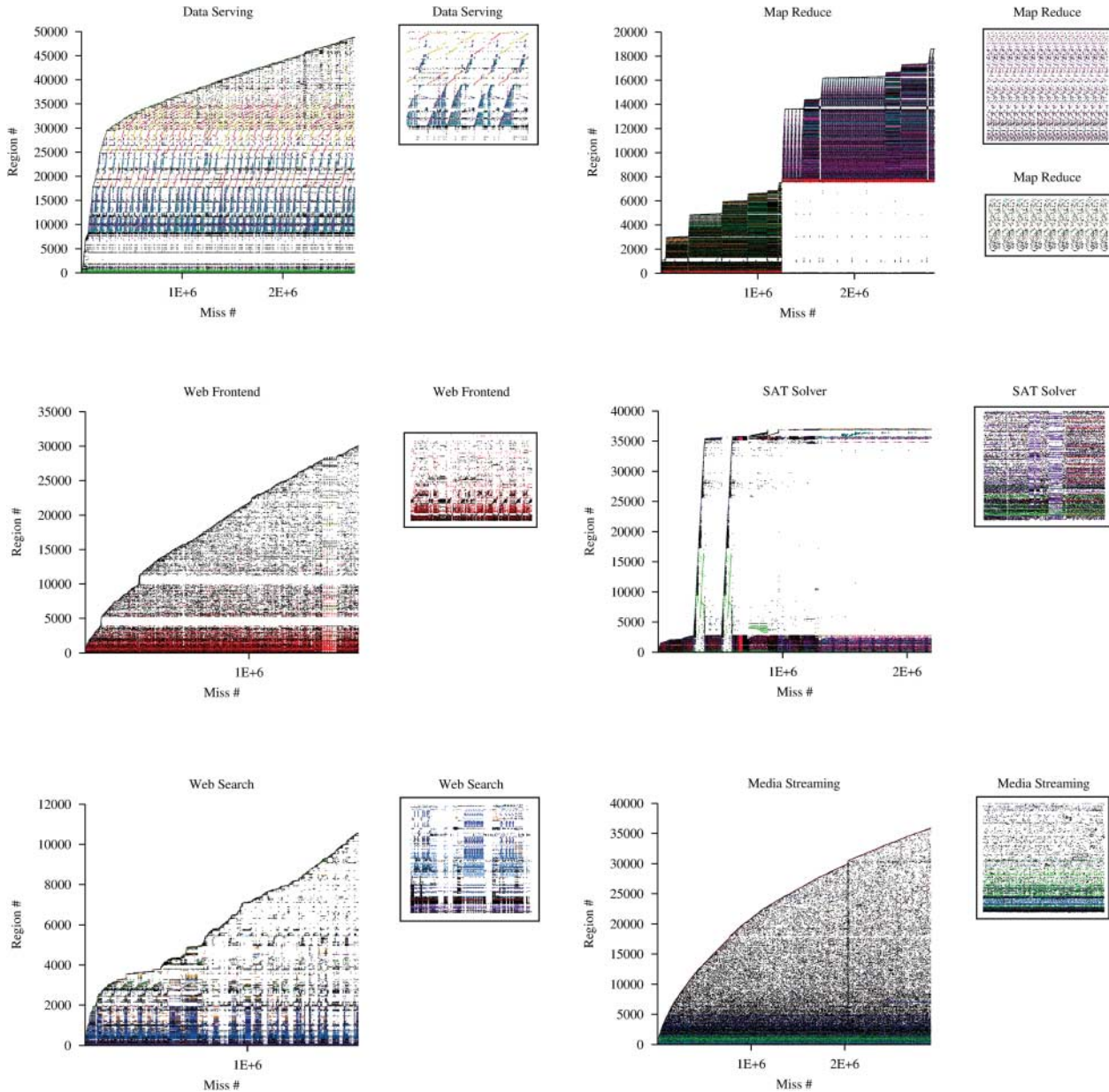


FIGURE 10. Repetition of misses of scale-out workloads. We can find recurring patterns of misses in some applications, like *Data Serving* and *Map Reduce* but patterns are complex. Recurring misses in other workloads are irregular so it is hard to find patterns in the history.

5.2. Time dimension

We use the Sequitur hierarchical data compression algorithm [33] to identify temporal repetitions in miss sequences. Sequitur builds a grammar whose production rules correspond to repetitions in its input. Each production rule assigns a label to a sequence of symbols and other rule labels. By reading one symbol at a time, Sequitur incrementally extends the grammar's root production rule. As each symbol is appended, the grammar is modified to create new production rules that capture any new repetition the appended symbol creates. As the

grammar grows, Sequitur maintains two constraints. First, no pair of symbols are adjacent more than once in the grammar. Second, every production rule in the grammar (except the root rule) is used more than once.

The primary goal of the Sequitur miss repetition analysis is to establish an upper bound for the opportunity to eliminate repetitive miss sequences via address correlation prefetching methods. We use the following categorization: *non-repetitive*: misses that do not recur; *new*: the first occurrence of a repetitive sequence; *head*: the first element in subsequent occurrences;

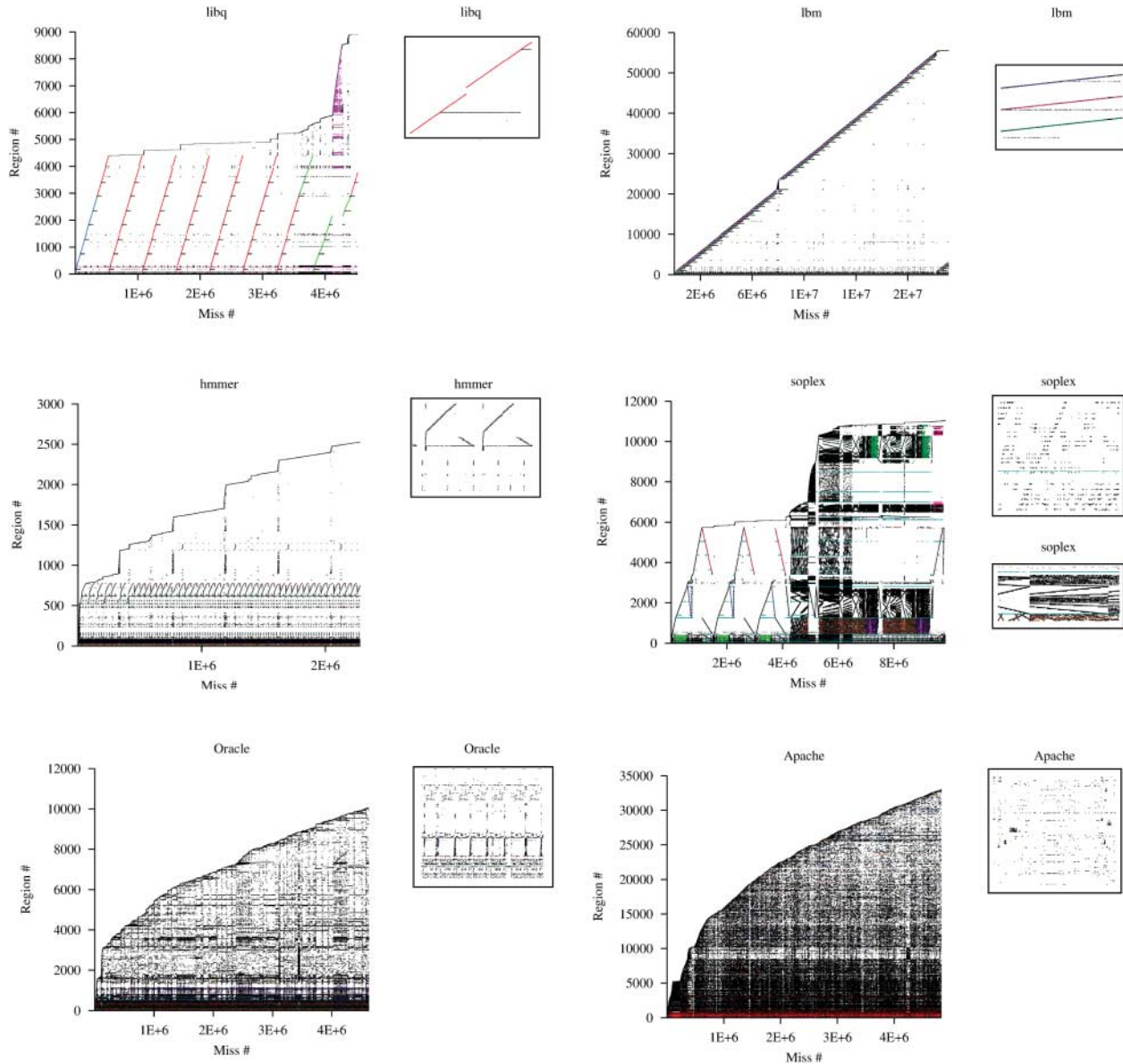


FIGURE 11. Repetition of misses in traditional workloads. SPEC workloads have simple recurring patterns except *soplex*. Also most of the misses belong to certain PCs. In server workloads, *Oracle* also have recurring patterns which are detectable.

and *opportunity*: non-head elements in repetitive occurrences of a sequence. Figure 12 shows a sequence with the corresponding Sequitur grammar. Since four symbols, p , q , r and s occur only in the root rule, they are considered as non-repetitive. The first occurrence of other symbols, w , x , y and z in the rules are classified as new. To find the head symbols, we have to find the first terminal symbol of all the previously encountered repeats among all possibilities. Therefore, the second occurrence of x and w are classified as heads. All other remaining symbols are then considered as opportunity.

Figure 13 shows the result of sequitur analysis for the workloads. Generally in Cloudsuite applications, except *Map Reduce*, there are little opportunities for eliminating repetitive misses (less than 50%). However, traditional server workloads have high opportunities (85% for *Oracle* and 60% for *Apache*). SPEC workloads have different behavior in a way that there are high opportunities to eliminate recurring misses in *libquantum* and *hammer* although they have higher miss rates than Cloudsuite workloads. These workloads also have strong and predictable miss patterns in space dimension (Fig. 11).

The opportunity in *lbm* is very low; however, this workload has predictable miss patterns in space (Fig. 11). In *soplex*, the opportunity for temporal recurring misses is less than 50%

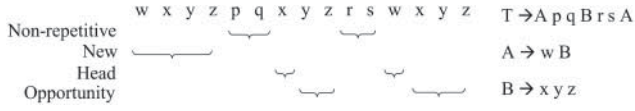


FIGURE 12. Example of Sequitur. A grammar is built from the sequence and the repetition of rules shows an opportunity to eliminate recurring misses.

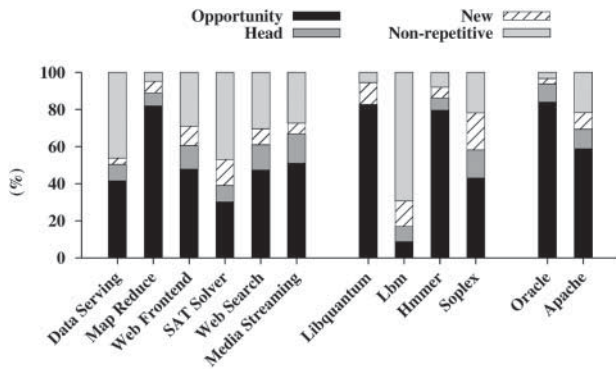


FIGURE 13. Sequitur analysis for workloads. In contrast to server workloads, a large fraction of misses in Cloudsuite workloads, except *Map Reduce*, are non-repetitive. Therefore, the opportunity to find repetitive miss patterns is low.

and spatial miss patterns are more predictable than Cloudsuite workloads.

5.3. Delta analysis

In this part, we analyze consecutive deltas for cache misses. The purpose of this analysis is to find out *inter-block* and *intra-block* transitions that lead to either cache hit or miss. Assume the block size is 64B which means that addresses in the range from 0 to 63 map to the first block, addresses in the range of 64 to 127 map to the second block, and so on. Now assume that there are two consecutive accesses to address 10 and address 11. That means the delta of 1 results in a hit because addresses 10 and 11 map to the first block. However, two consecutive accesses to 63 and 64 (delta is 1) results in a miss. Therefore, a delta may result in hit or miss. In cache-friendly applications, it is expected that a few percent of frequently repeated deltas result in cache miss. To find the frequently repeated deltas, all deltas are sorted in a list based on their repetitions in descending order. As a result, the first entry is a delta with the highest repetition score and the last entry is a delta with the lowest repetition score.

Figure 14a shows an example which is related to cache-friendly applications. Mostly reused delta which is 2, repeats six times and only one of its occurrences results in a miss. A delta of four repeats four times and all of its occurrences result in hits. Each of the two least repeated deltas, e.g. 10 and 3, repeats three times and results in a miss. Plotting the miss ratios based on the order of repeated deltas (and not their values) shows that the concentration of data points is on low miss

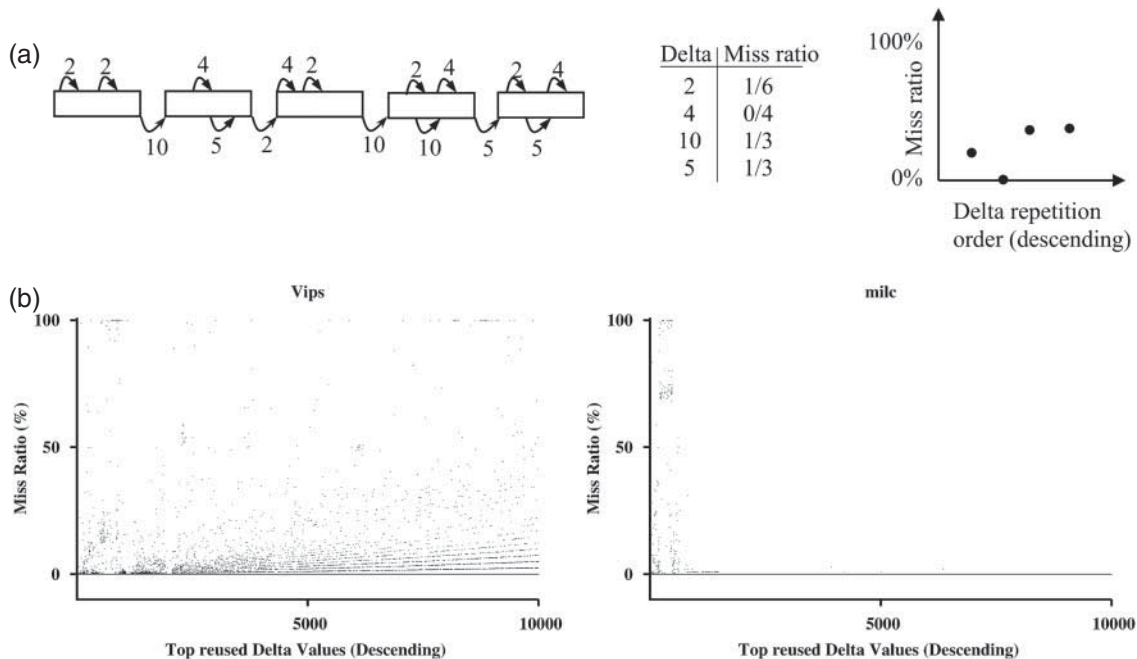


FIGURE 14. Effect of top reused deltas on cache misses. (a) An example to show that in cache-friendly applications, top reused deltas mostly hit in cache and (b) the result for *Vips* and *Milc*.

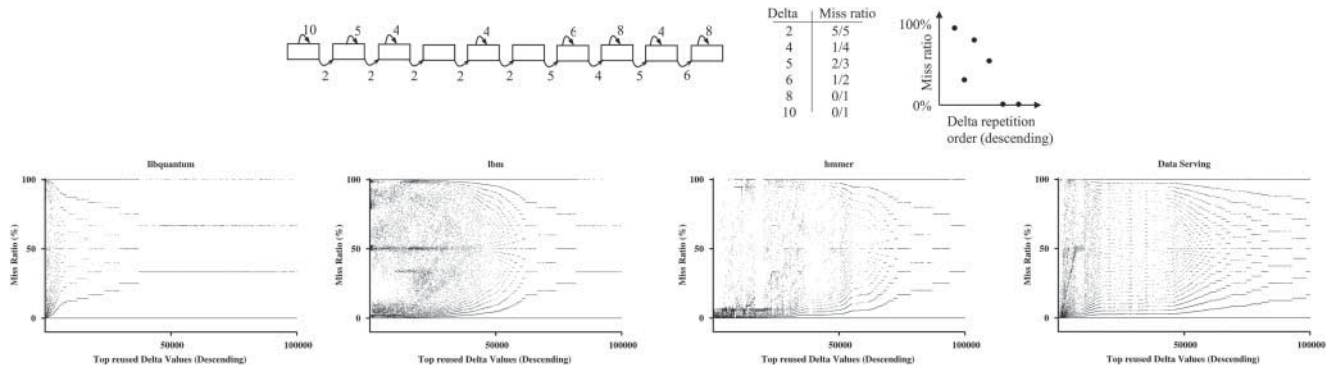


FIGURE 15. Effect of top reused deltas on cache misses for prefetch-friendly applications. Since delta values frequently repeat, they converge quickly. Therefore, delta-based prefetcher are able to find and match patterns.

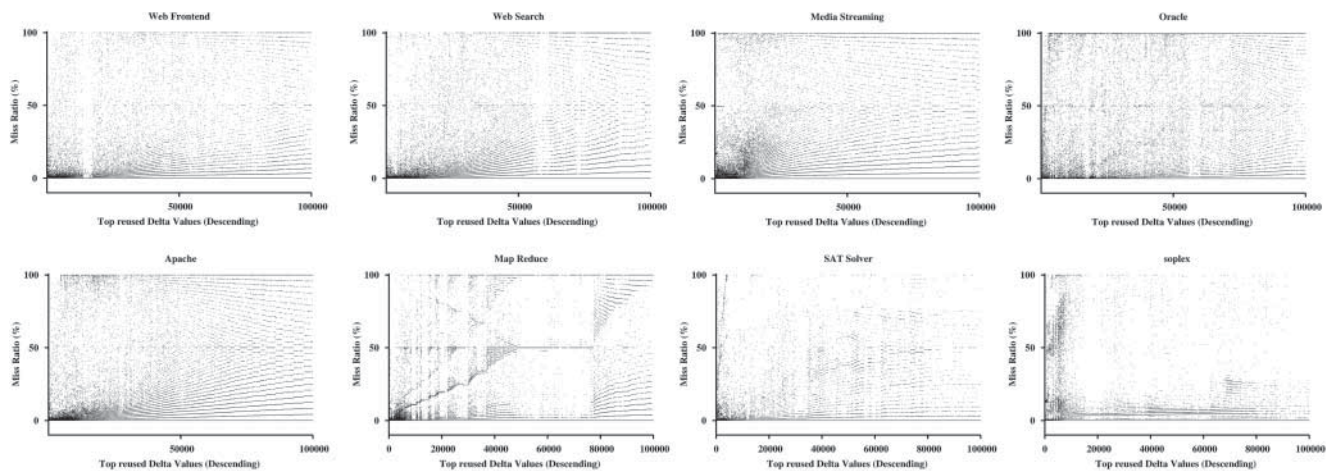


FIGURE 16. Effect of top reused deltas on cache misses in some non-cache-friendly applications. The results for most of the modern workloads show that the convergence takes place far beyond the limit.

ratios. Evaluating cache-friendly workloads, *Vips* and *Milc*, with low miss rates shows that the concentration is on low miss ratios as expected (Fig. 14b).

However, not all applications are cache-friendly. Assume that the most frequent delta is repeated m times. Then, the number of misses is variable between 0 to m . For a large value of m , the miss ratio takes any arbitrarily number between 0 and 100%. Moving toward the less repeated deltas, the number of possible miss ratios decreases. For example, the miss ratio of a delta which repeats four times can be either 0, 25, 50, 75 or 100%. Similarly, the miss ratio of a delta which repeats three times can be 0, 33, 66 or 100% and the miss ratio of a delta which repeats twice can be 0, 50 or 100%. Therefore, there exists a convergence as we move to the least repeated deltas. If such a convergence occurs quickly, that means deltas are highly repeatable. Otherwise, there is a large volume of deltas with few repetitions.

An example of such behavior is presented in Fig. 15. Most of the deltas frequently repeat, and hence they quickly converge. Applications with this property are suitable for delta-based

prefetchers (e.g. GHB PC/DC or stride prefetcher) because with a limited storage, it is possible to track recurring deltas. The result of this analysis on *Libquantum*, *Hammer* and *Lbm* shows that the convergence takes place within a given number of deltas.⁵ Also, the coverage of GHB PC/DC for these workloads confirms that they are suitable for delta-based prefetchers. *Data Serving* from the Cloudsuite is to some extent similar to SPEC workloads. However, the convergence occurs beyond the 100K limit and the result of GHB PC/DC confirms that it works better than other history-based prefetchers.

Figure 16 shows the results of this analysis on other workloads. As can be seen in web-related workloads (*Web Frontend*, *Web Search* and *Apache*), *Media Streaming* and *Oracle*, the convergence takes place far beyond the 100K limit. That means there are a large number of deltas with few repetitions. *Map Reduce* and *SAT Solver* from Cloudsuite have random and

⁵ To better illustrate the results, we limit the horizontal axis to 100K most frequent deltas.

unstructured shapes and the coverage of GHB PC/DC shows delta-based prefetchers are not able to find delta patterns of these workloads. Although *Soplex* from SPEC CPU has an unstructured shape, the number of repeated deltas is high and the concentration is on low miss ratios.

As a summary, by comparing modern workloads with traditional SPEC workloads, we see that they have a huge number of distinct delta values. Therefore, a history-based prefetcher that tracks delta values has to store a large volume of deltas to find a recurring pattern which means a large table should be incorporated. Additionally, delta values randomly hit or miss in the cache which cause the prefetcher to incorrectly find matching patterns in the history and mispredict future addresses.

6. ENERGY ASPECTS

Apart from static energy which is consumed by the on-chip storages in a prefetcher, it consumes dynamic energy due to accessing lower levels of memory hierarchy. Fact is, every prefetch message is considered as load request, so it is issued to the lower memory levels to bring the data back to the L1 but it is not delivered to the pipeline. Analyses using CACTI and McPAT show that components in lower levels including last-level cache (LLC) slices, routers, directories and main

memory dissipate energies approximately $10\times$, $15\times$, $1.2\times$ and $100\times$ the L1 data cache, respectively.

With the poor coverages achieved, this is an important problem. Table 3 shows a brief survey on the increase in memory requests. Note that for each prefetching method and workload, we count the number of memory requests due to the demand misses, i.e. misses caused by load instructions, and prefetch requests. Then, we normalized the value to the baseline architecture with no prefetcher. Finally, we calculated the average number of memory requests over the scale-out workloads. As can be seen, the increase in the memory requests are much higher than the coverages. Therefore, using prefetchers is not energy efficient.

7. RELATED WORK

To our best knowledge, no data prefetcher has been proposed specifically for modern workloads which tries to hide the miss latency at L1 data cache. We claim that this is the first work that evaluates the in-depth aspects of L1 data cache for emerging cloud and big data workloads. Table 4 shows a summary of related works with the scopes and workloads that have been used in their evaluations. Nearly all data prefetching proposals in recent years are dedicated to traditional desktop workloads (namely SPEC CPU suite). There are some limited works on traditional and commercial server workloads, e.g. OLTP and web applications. Additionally, there are few number of works on instruction prefetching for emerging cloud workloads. As mentioned before, recent data prefetching proposals evaluate their algorithms with Cloudsuite. Ferdman [21] also showed that disabling built-in next-line prefetcher which is available in commercial processors does not degrade the performance for Cloudsuite. Pugsley [17] have also evaluated sandbox prefetching for multi-program sets of SPEC CPU and Cloudsuite workloads at the last-level-cache. Their results confirm that the prefetcher performance is negligible for Cloudsuite

TABLE 3. Average coverage and increase in memory request for scale-out workloads.

	Coverage (%)	Memory requests (%)
NLP	24.16	83.28
Stream (med)	13.03	90.69
GHB (PC/DC)	9.2	92.28
ISB	-5.4	92.4
SMS	19.5	33.9

TABLE 4. Related works to prefetching and locality analysis, their scope and workloads used in their evaluation. Most of the works on cache including locality and data prefetching methods are done for traditional workloads.

Related work	Scope	Workloads used
Next-line [3], Stride [6], AMPM [10], GHB [4], Markov [7], FDP [8], ISB [18], stream [27], Palacharla's adaptive prefetcher [26]	Data prefetcher	SPEC CPU
STeMS [20], SMS [19], TMS [36]	Data prefetcher	Traditional servers
PIF [37], SHIFT [38]	Instruction prefetcher	Traditional servers and cloud
Locality analysis using reuse distance [39], Predicting whole-program locality [29], Cache management policies with reuse distances [30], Optimal multicore cache hierarchies via reuse distances [31], Cache replacement with reuse distances [32], Temporal and Spatial locality measurements [34], Locality measurements for HPC [35], Component model of spatial locality [40]	Locality analyses	SPEC CPU and HPC
Sandbox [17], Clearing the clouds [21]	Data prefetcher	SPEC CPU and cloud (partially)

workloads compared with traditional SPEC CPU workloads. None of these works discusses on why prefetchers fail to increase the performance of modern workloads. Also, they have not justified why the used prefetcher (next-line or sandbox) has no impact on the performance.

Evaluating prefetching methods and their feasibility are partially done for HPC workloads. Lee [15] have studied why prefetching does not work for some of the engineering workloads (SPEC CPU) by explaining part of the source codes that make the access patterns irregular.

The principal of locality has long been a fundamental research subject but most of the related works in this subject try to evaluate the locality principal for HPC applications to group them and estimate the miss rates with probabilistic methods. However, this work is orthogonal to them since we know the miss ratios of modern workloads.

Dummler [34] have extended the spatial locality measurement to make a platform independent decision. As a result, a more accurate prediction on miss rates are possible especially for HPC applications. The extension basically considers the accesses to physical neighboring memory locations. Yu [35] have proposed a new metric, miss bandwidth, to measure both temporal and spatial localities for HPC applications. Miss bandwidth reflects the amount of memory traffic generated by misses of a fully associative cache. With a good temporal locality, the miss bandwidth decreases rapidly as a function of cache size. Similarly, if the miss bandwidth is increased by a factor of cache line size, that means the program features poor spatial locality.

Gu [40] have proposed a component-based model for spatial locality of SPEC CPU2000 applications by measuring the changes of reuse distance as a function of data block size. The spatial locality score is then built from a reuse signature and tracking the reuse distance. Similarly, locality analysis for predicting the behavior is done in [29, 39]. Reuse distance does not model the repetitions accurately because two different addresses may have the same reuse distance.

In [41], the authors have studied the temporal address correlation in commercial server workloads. Unlike temporal locality which considers the repetition of one address, temporal correlation deals with a group of shared addresses that tend to be accessed together in the same order. It has been shown that this feature exists and there are opportunities to hide the memory latency by prefetching future blocks. In a similar work [36], they have used Sequitur, a hierarchical data compression algorithm to identify the repetitive subsequences within the miss traces.

8. SUMMARY AND CONCLUSION

This paper was studied cloud workloads to show that unlike traditional workloads, prefetching methods are neither effective for hiding memory latency nor efficient for saving energy

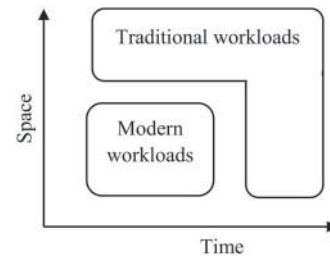


FIGURE 17. Contrary to traditional server and desktop workloads, emerging cloud workloads have irregular miss pattern over the memory space and the opportunity to find recurring miss patterns over the time is low.

for such workloads. Results showed that although there exists a number of traditional workloads that have higher miss rates than modern workloads, prefetchers fail to correctly predict future data accesses and most of the prefetched blocks are evicted from the cache without any use. The summary of this study can be seen in Fig. 17. Traditional desktop and engineering workloads (SPEC CPU) have either high temporal and spatial miss patterns or predictable spatial miss patterns when the opportunity to remove repetitive misses is low. In order to increase the performance of caches, one has to incorporate very large on-chip caches to capture the accesses. This is not a practical solution since in spite of technological limitations on putting a large cache on-chip (due to area and power constraints), the access latency is increased. Manufacturers, as shown in Fig. 1, tend to use small caches to keep access latency as low as possible and at the same time they use data prefetchers (shown to be useful for traditional workloads) to compensate incurred misses. Our analysis showed that state-of-the-art prefetchers make incorrect predictions because they are tracking a small regions (a small L1 cache window which monitors memory accesses temporally and spatially). Therefore, in order to design a prefetcher for modern workloads, it has to be able to track long term accesses without losing the accuracy by dismissing short-term accesses. Such a prefetcher needs a large storage which is again not practical with today's technology. For this reason and to reduce energy consumption due to superfluous accesses to the lower levels of memory hierarchy, commercial processors targeting cloud and big data workloads should not use data prefetchers. Although the focus of this paper was the L1 data cache, we are studying miss patterns in other cache types, e.g. L2 and L3 caches, along with software code models to figure out the opportunities for prefetching at other levels.

REFERENCES

- [1] Callahan, D., Kennedy, K. and Porterfield, A. (1991) Software Prefetching. *Proceedings of Architectural Support for Programming Languages and Operating Systems*, Santa Clara, USA, April 8–11, pp. 40–52. ACM, New York.

- [2] Klaiber, A. C. and Levy, H. M. (1991) An Architecture for Software-controlled Data Prefetching. *Proceedings International Symposium on Computer Architecture*, Toronto, Canada, May 27–30, pp. 43–53. ACM, New York.
- [3] Smith, A. J. (1982) Cache memories. *ACM Comput. Surveys*, **14**, 473–530.
- [4] Nesbit, K. J. and Smith, J. E. (2004) Data Cache Prefetching using a Global History Buffer. *Proceedings International Symposium on High Performance Computer Architecture*, Madrid, Spain, February 14–18, pp. 96–106. IEEE, Washington DC.
- [5] Nesbit, K. J., Dhodapkar, S. and Smith, J. E. (2004) Ac/dc: An Adaptive Data Cache Prefetcher. *Proceedings of International Conference on Parallel Architectures and Compilation Techniques*, Antibes Juan-les-Pins, France, September 29–October 3, pp. 135–145. IEEE, Washington DC.
- [6] Baer, J.-L. and Chen, T.-F. (1995) Effective hardware-based data prefetching for high-performance processors. *IEEE Trans. Comput.*, **44**, 609–623.
- [7] Joseph, D. and Grunwald, D. (1997) Prefetching using Markov Predictors. *Proceedings of International Symposium on Computer Architecture*, Denver, USA, June 2–4, pp. 252–263. ACM, New York.
- [8] Srinath, S., Mutlu, O., Kim, H. and Patt, Y. N. (2007) Feedback Directed Prefetching: Improving the Performance and Bandwidth-efficiency of Hardware Prefetchers. *Proceedings of International Symposium on High Performance Computer Architecture*, Phoenix, USA, February 10–14, pp. 63–74. IEEE, Washington DC.
- [9] Perez, D. G., Mouchard, G. and Temam, O. (2004) Microlib: A Case for the Quantitative Comparison of Micro-architecture Mechanisms. *Proceedings of International Symposium on Microarchitecture*, Portland, USA, December 4–8, pp. 43–54. IEEE, Washington DC.
- [10] Ishii, Y., Inaba, M. and Hiraki, K. (2009) Access Map Pattern Matching for Data Cache Prefetch. *Proceedings of International Conference on Supercomputing*, Yorktown Heights, USA, June 8–12, pp. 499–500. ACM, New York.
- [11] Ebrahimi, E., Mutlu, O., Lee, C. J. and Patt, Y. N. (2009) Coordinated Control of Multiple Prefetchers in Multi-core Systems. *Proceedings of International Symposium on Microarchitecture*, New York, USA, December 12–16, pp. 316–326. IEEE, Washington DC.
- [12] Ebrahim, E., Lee, C. J., Mutlu, O. and Patt, Y. N. (2011) Prefetch-aware Shared Resource Management for Multi-core Systems. *Proceedings of International Symposium on Computer Architecture*, San Jose, USA, June 4–8, pp. 141–152. ACM, New York.
- [13] Lee, C. J., Narasiman, V., Mutlu, O. and Patt, Y. N. (2009) Improving Memory Bank-level Parallelism in the Presence of Prefetching. *Proceedings of International Symposium on Microarchitecture*, New York, USA, December 12–16, pp. 327–336. IEEE, Washington DC.
- [14] Ebrahimi, E., Mutlu, O. and Patt, Y. N. (2009) Techniques for Bandwidth-efficient Prefetching of Linked Data Structures in Hybrid Prefetching Systems. *Proceedings of International Symposium on High Performance Computer Architecture*, Raleigh, USA, February 14–18, pp. 7–17. IEEE, Washington DC.
- [15] Lee, J., Kim, H. and Vuduc, R. (2012) When prefetching works, when it does not, and why. *ACM Trans. Archit. Code Optim.*, **9**, 2:1–2:29.
- [16] Lee, C. J., Mutlu, O., Narasiman, V. and Patt, Y. N. (2011) Prefetch-aware memory controllers. *IEEE Trans. Comput.*, **60**, 1406–1430.
- [17] Pugsley, S. H., Chishti, Z., Wilkerson, C., Chuang, P.-F., Scott, R. L. and Jaleel, A. (2014) Sandbox Prefetching: Safe, Run-time Evaluation of Aggressive Prefetchers. *Proceedings of International Symposium on High Performance Computer Architecture*, Orlando, USA, February 15–19, pp. 626–637. IEEE, Washington DC.
- [18] Jain, A. and Lin, C. (2013) Linearizing Irregular Memory Accesses for Improved Correlated Prefetching. *Proceedings of International Symposium on Microarchitecture*, Davis, USA, December 7–11, pp. 247–259. ACM, New York.
- [19] Somogyi, S., Wenisch, T. F., Ailamaki, A., Falsafi, B. and Moshovos, A. (2006) Spatial Memory Streaming. *Proceedings of International Symposium on Computer Architecture*, Boston, USA, June 17–21, pp. 252–263. IEEE, Washington DC.
- [20] Somogyi, S., Wenisch, T. F., Ailamaki, A. and Falsafi, B. (2009) Spatio-temporal Memory Streaming. *Proceedings International Symposium on Computer Architecture*, Austin, USA, June 20–24, pp. 69–80. ACM, New York.
- [21] Ferdman, M., Adileh, A., Kocberber, O., Volos, S., Alisafae, M., Jevdjic, D., Kaynak, C., Popescu, A. D., Ailamaki, A. and Falsafi, B. (2012) Clearing the Clouds: A Study of Emerging Scale-out Workloads on Modern Hardware. *Proceedings of Architectural Support for Programming Languages and Operating Systems*, London, UK, March 3–7, pp. 37–48. ACM, New York.
- [22] Hardavellas, N., Somogyi, S., Wenisch, T. F., Wunderlich, E., Chen, S., Kim, J., Falsafi, B., Hoe, J. C. and Nowatzky, A. G. (2004) Simflex: a fast, accurate, flexible full-system simulation framework for performance evaluation of server architecture. *ACM SIGMETRICS Perform. Eval. Rev.*, **31**, 31–34.
- [23] Bienia, C., Kumar, S., Singh, J. P. and Li, K. (2008) The PARSEC Benchmark Suite: Characterization and Architectural Implications. *Proceedings of International Conference on Parallel Architectures and Compilation Techniques*, Toronto, Canada, October 25–29, pp. 72–81. ACM, New York.
- [24] Henning, J. L. (2006) Spec cpu2006 benchmark descriptions. *ACM SIGARCH Comput. Archit. News*, **34**, 1–17.
- [25] Tendler, J. M., Dodson, J. S., Fields, J. S., Le, H. and Sinharoy, B. (2002) Power4 system microarchitecture. *IBM J. Res. Dev.*, **46**, 5–25.
- [26] Palacharla, S. and Kessler, R. E. (1994) Evaluating Stream Buffers as a Secondary Cache Replacement. *Proceedings of International Symposium on Computer Architecture*, Chicago, USA, April 18–21, pp. 24–33. IEEE Computer Society Press, Los Alamitos, CA.
- [27] Jouppi, N. P. (1990) Improving Direct-mapped Cache Performance by the Addition of a Small Fully-associative Cache and Prefetch Buffers. *Proceedings of International Symposium on Computer Architecture*, Seattle, USA, May 28–31, pp. 364–373. ACM, New York.
- [28] Mattson, R. L., Gecsei, J., Slutz, D. R. and Traiger, I. L. (1970) Evaluation techniques for storage hierarchies. *IBM Syst. J.*, **9**, 78–117.

- [29] Ding, C. and Zhong, Y. (2003) Predicting Whole-program Locality through Reuse Distance Analysis. *Proceedings of SIGPLAN Conference on Programming Language Design and Implementation*, San Diego, CA, June 9–11, pp. 245–257. ACM, New York.
- [30] Duong, N., Zhao, D., Kim, T., Cammarota, R., Valero, M. and Veidenbaum, A. V. (2012) Improving Cache Management Policies using Dynamic Reuse Distances. *Proceedings of International Symposium on Microarchitecture*, Vancouver, Canada, December 1–5, pp. 389–400. IEEE, Washington DC.
- [31] Wu, M. J. and Yeung, D. (2012) Identifying Optimal Multicore Cache Hierarchies for Loop-based Parallel Programs Via Reuse Distance Analysis. *Proceedings of SIGPLAN Workshop on Memory Systems Performance and Correctness*, Beijing, China, June 11–16, pp. 2–11. ACM, New York.
- [32] Keramidas, G., Petoumenos, P. and Kaxiras, S. (2007) Cache Replacement based on Reuse-distance Prediction. *Proceedings of International Conference on Computer Design*, Lake Tahoe, USA, October 7–10, pp. 245–250. ACM, New York.
- [33] Nevill-Manning, C. G. and Witten, I. H. (1997) Identifying hierarchical structure in sequences: a linear-time algorithm. *J. Artif. Intell. Res.*, **7**, 67–82.
- [34] Dummler, J., Rauber, T. and Runger, G. (2006) Combining Measures for Temporal and Spatial Locality. *Proceedings of International Conference Frontiers of High Performance Computing and Networking*, Sorrento, Italy, December 4–7, pp. 697–706. Springer, Berlin.
- [35] DCS-R-2005-2564 (2005) *A New Locality Metric and Case Studies for HPCS Benchmark*. University of Illinois, Urbana Champaign, Champaign, IL.
- [36] Wenisch, T. F., Ferdman, M., Ailamaki, A., Falsafi, B. and Moshovos, A. (2008) Temporal Streams in Commercial Server Applications. *Proceedings of International Symposium on Workload Characterization*, Seattle, USA, September 14–16, pp. 99–108. IEEE, Washington DC.
- [37] Ferdman, M., Kaynak, C. and Falsafi, B. (2011) Proactive Instruction Fetch. *Proceedings of International Symposium on Microarchitecture*, Porto Alegre, Brazil, December 4–7, pp. 152–162. ACM, New York.
- [38] Kaynak, C., Grot, B. and Falsafi, B. (2013) Shift: Shared History Instruction Fetch for Lean-core Server Processors. *Proceedings of International Symposium on Microarchitecture*, Davis, USA, December 7–11, pp. 272–283. ACM, New York.
- [39] Zhong, Y., Shen, X. and Ding, C. (2009) Program locality analysis using reuse distance. *ACM Trans. Program. Lang. Syst.*, **31**, 20:1–20:39.
- [40] Gu, X., Christopher, I., Bai, T., Zhang, C. and Ding, C. (2009) A Component Model of Spatial Locality. *Proceedings of International Symposium on Memory Management*, Dublin, Ireland, June 19–20, pp. 99–108. ACM, New York.
- [41] Wenisch, T. F., Somogyi, S., Hardavellas, N., Kim, J., Ailamaki, A. and Falsafi, B. (2005) Temporal Streaming of Shared Memory. *Proceedings of International Symposium on Computer Architecture*, Madison, USA, June 4–8, pp. 222–233. ACM, New York.